COLA: 基于多智能体的人机协同景观设计系统

1. TLDR

1.1 Definition 产品定义

COLA, short for COpilot for Landscape Architecture, is a human-in-the-loop multi-agent system for Landscape Architecture design.

COLA是COpilot for Landscape Architecture的缩写，是基于多智能体的人机协同景观设计系统。

1.2 Positioning 产品定位

Timing: Early Design Stage
Scenario: Idea exchange and alignment of design proposals with clients, brainstorming, and rapid prototyping.

时间：设计前期阶段

场景：与客户的设计方案想法碰撞和对齐、头脑风暴和快速原型设计。

2. Problem/Pain-point & Solution 存在问题/市场痛点 & 解决方案

2.1 景观设计专业性

- LLM
  - 方案规划 ---> RAG相似案例
  - 提示词用词 ---> RAG专业词库
- VLM
  - 平面图效果 ---> LORA微调模型

3. Abstraction 概念抽象

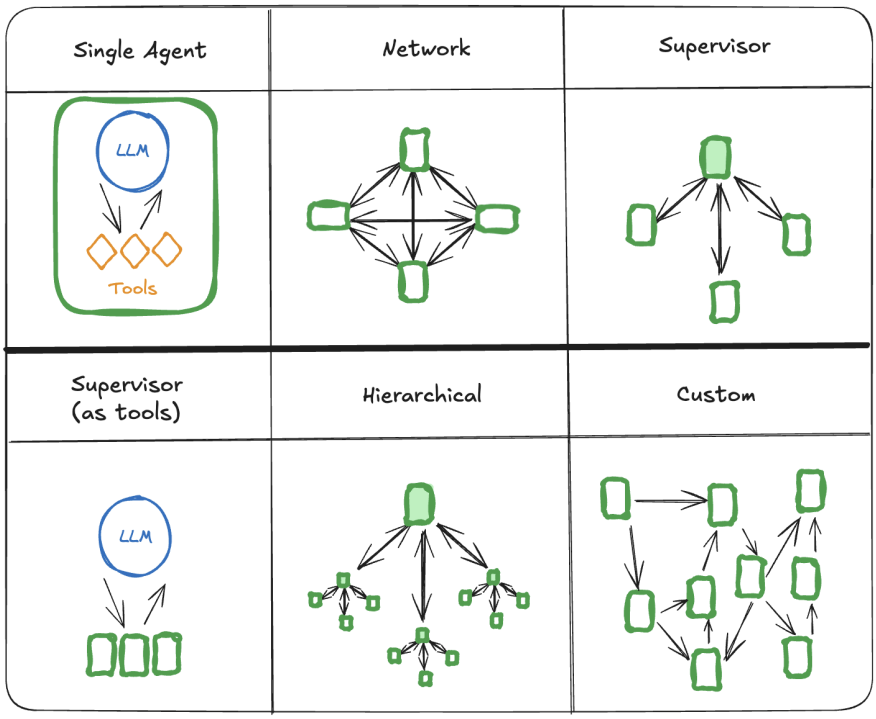
3.1 Formalized function 形式化表述

COLA={Supervisor, Planner, Adapter, Renderer, Knowledge-base}

COLA={主管、计划器、适配器、渲染器、知识库}

3.2 Multi-agent architecture 多智能体架构

3.2.1 Multi-agent architectures 多智能体架构



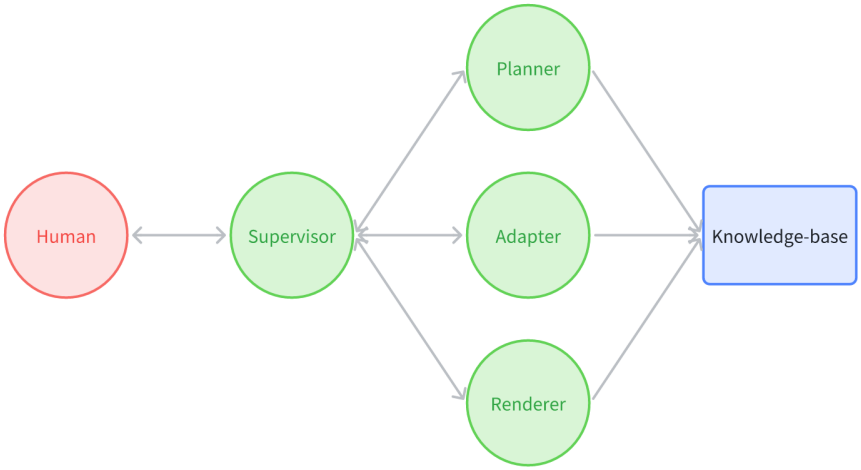
- **Supervisor**: each agent communicates with a single supervisor agent. Supervisor agent makes decisions on which agent should be called next.

  主管：每个agent都与一个主管agent进行通信。主管agent决定接下来应该调用哪个agent。

- **Supervisor (tool-calling)**✅: this is a special case of supervisor architecture. Individual agents can be represented as tools. In this case, a supervisor agent uses a tool-calling LLM to decide which of the agent tools to call, as well as the arguments to pass to those agents.

  主管（工具调用）：这是主管架构的一种特殊情况。单个agent可以表示为工具。在这种情况下，主管agent使用工具调用LLM来决定调用哪些agent工具，以及传递给这些agent的参数。

3.2.2 COLA architectures COLA架构



- "SPARK"
- Supervisor: LLM
- Planner: LLM + RAG
- Adapter: LLM + RAG
- Renderer: VLM + LORA
- Knowledge-base: Web-data + Local-data

3.3 Agent architectures 智能体架构

ReAct(ReAct: Synergizing Reasoning and Acting in Language Models) is a popular general purpose agent architecture that combines these expansions, integrating three core concepts.

a. `Planning` : Empowering the LLM to create and follow multi-step plans to achieve goals

b. `Tool calling` : Allowing the LLM to select and use various tools as needed.

c. `Memory` : Enabling the agent to retain and use information from previous steps.

ReAct（ReAct：语言模型中的协同推理和行动）是一种流行的通用agent架构，它结合了这些扩展，集成了三个核心概念。

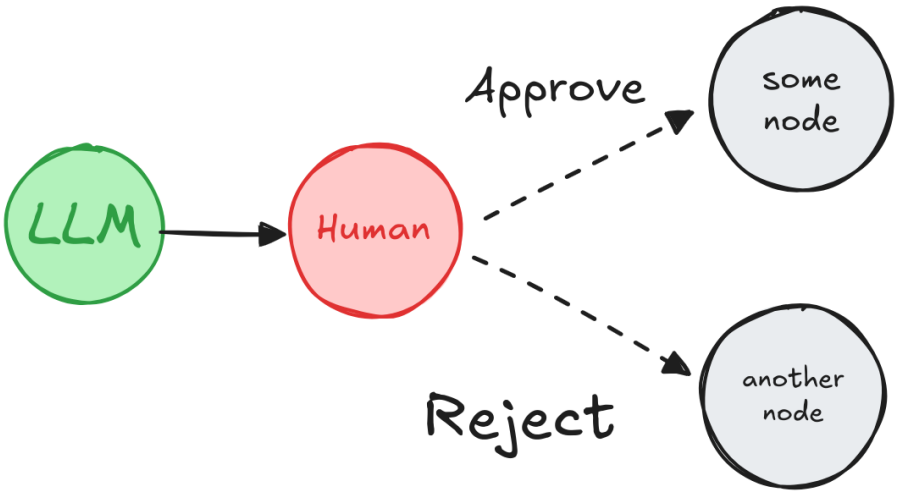
a. 规划：授权法学硕士创建和遵循多步骤计划以实现目标

b. 工具调用：允许LLM根据需要选择和使用各种工具。

c. 记忆：使agent能够保留和使用来自先前步骤的信息。

3.4 HITP Design Patterns 人机交互设计模式

3.4.1 Approve or Reject 批准或拒绝✅



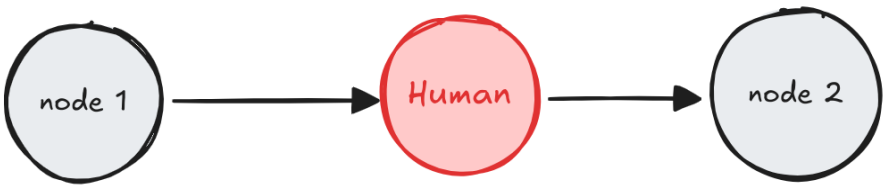
Depending on the human's approval or rejection, the graph can proceed with the action or take an alternative path.

根据人类的批准或拒绝，图表可以继续执行操作或采取替代路径。

Pause the graph before a critical step, such as an API call, to review and approve the action. If the action is rejected, you can prevent the graph from executing the step, and potentially take an alternative action.

在关键步骤（如API调用）之前暂停图形以审查和批准操作。如果操作被拒绝，您可以阻止图形执行该步骤，并可能采取替代操作。

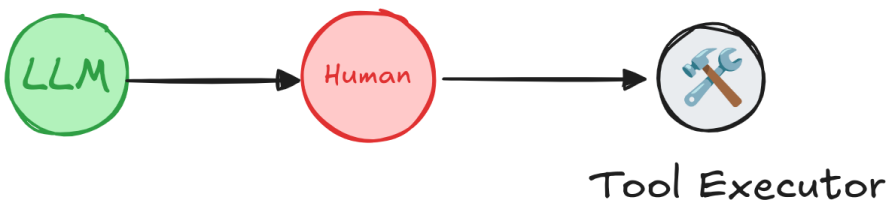
3.4.2 Review & Edit State 查看和编辑状态✅



A human can review and edit the state of the graph. This is useful for correcting mistakes or updating the state with additional information.

人类可以查看和编辑状态，这对于纠正错误或使用附加信息更新状态非常有用。

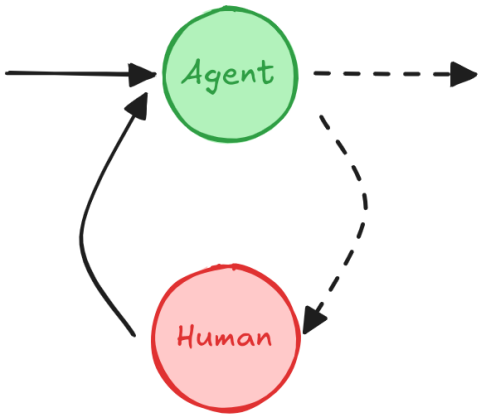
3.4.3 Review Tool Calls 审查工具调用✅



A human can review and edit the output from the LLM before proceeding. This is particularly critical in applications where the tool calls requested by the LLM may be sensitive or require human oversight.

人类可以在继续之前审查和编辑LLM的输出，这在LLM请求的工具调用可能敏感或需要人工监督的应用程序中尤为关键。

3.4.4 Multi-turn conversation 多轮对话✅



A **multi-turn conversation** architecture where an **agent** and **human node** cycle back and forth until the agent decides to hand off the conversation to another agent or another part of the system.

一种多轮对话架构，其中agent和人类节点来回循环，直到agent决定将对话移交给另一个agent或系统的另一部分。

A **multi-turn conversation** involves multiple back-and-forth interactions between an agent and a human, which can allow the agent to gather additional information from the human in a conversational manner.

多轮对话涉及agent和人类之间的多次来回交互，这可以允许agent以对话方式从人类那里收集附加信息。

This design pattern is useful in an LLM application consisting of multiple agents. One or more agents may need to carry out multi-turn conversations with a human, where the human provides input or feedback at different stages of the conversation.

这种设计模式在由多个agent组成的LLM应用程序中很有用。一个或多个agent可能需要与人类进行多轮对话，其中人类在对话的不同阶段提供输入或反馈。

3.4.4.1 Using a human node per agent 每个agent使用一个人工节点

In this pattern, each agent has its own human node for collecting user input. This can be achieved by either naming the human nodes with unique names (e.g., "human for agent 1", "human for agent 2") or by using subgraphs where a subgraph contains a human node and an agent node.

在这种模式中，每个agent都有自己的人类节点来收集用户输入。这可以通过使用唯一名称（例如，“人类代表agent1”，“人类代表agent2”）或使用子图来实现，其中子图包含人类节点和agent节点。

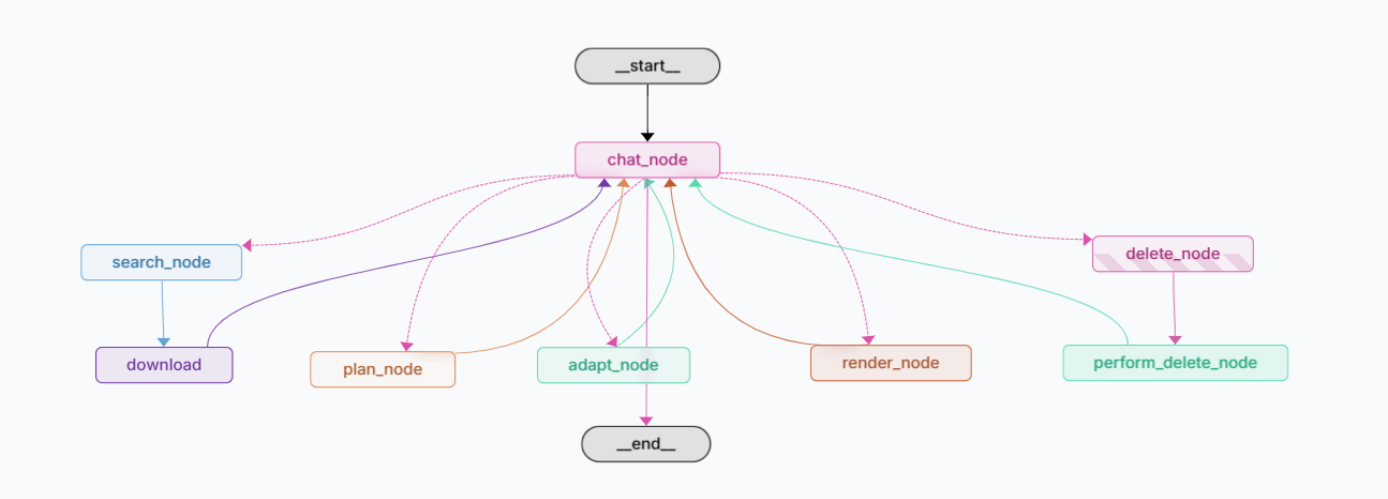
3.4.4.2 Sharing a human node across multiple agents 跨多个agent共享一个人工节点✅

In this pattern, a single human node is used to collect user input for multiple agents. The active agent is determined by the state, so after human input is collected, the graph can route to the correct agent.

在这种模式中，单个人工节点用于收集多个agent的用户输入。活动agent由状态决定，因此在收集到人工输入后，图可以路由到正确的agent。

4. Implementation 代码实现

4.1 概览



4.2 代码结构与技术

4.2.1 后端

- System: FastAPI, CopilotKit
- Agents: LangGraph
- LLM: DeepSeek(V3)
- VLM: StableDiffusion(Flux.1)
- LORA: LiblibAI

4.2.2 前端

- System: React, CopilotKit

5. Future Work 未来工作

5.1 RAG

5.2 LORA

5.3 System

5.4 Paper

6. Reference 参考资料

6.1 Multi-agent Systems

https://langchain-ai.github.io/langgraph/concepts/multi_agent/

6.2 Agent architectures

https://langchain-ai.github.io/langgraph/concepts/agentic_concepts/

6.3 Human-in-the-loop

https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/#multi-turn-conversation